Занятие 9: Транзакции и индексы

Атомарность, конкурентные изменения, оптимизация

Слайд 1: Что такое транзакция?

Транзакция - группа операций, выполняющихся как единое целое

Свойства ACID:

- **A**tomicity - атомарность (все или ничего)
- **C**onsistency - согласованность
- **I**solation - изоляция
- **D**urability - долговечность

Пример:

```
Перевод денег:
1. Снять 100$ со счёта A
2. Добавить 100$ на счёт B

Если шаг 2 не выполнится → шаг 1 должен быть отменён
```

Слайд 2: BEGIN, COMMIT, ROLLBACK

Базовые команды транзакции:

```
BEGIN;  -- Начать транзакцию

-- Операции
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;  -- Подтвердить изменения
```

Если ошибка:

```
BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
-- Ошибка на следующей операции
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  -- FAIL

ROLLBACK;  -- Отменить все изменения
```

Слайд 3: Транзакция в Python

Пример с psycopg2:

```
conn = psycopg2.connect("...")
cursor = conn.cursor()

try:
    # Начать транзакцию (автоматически)
    cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")

    # Подтвердить
    conn.commit()

except Exception as e:
    # Отменить при ошибке
    conn.rollback()
    raise e

finally:
    cursor.close()
    conn.close()
```

Важно:

- `commit()` сохраняет изменения
- `rollback()` отменяет изменения
- Без `commit()` изменения не сохранятся

Слайд 4: Атомарность - зачем нужна?

Пример без транзакции:

```
# Шаг 1
cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
# Баланс уменьшен

# Шаг 2 - ОШИБКА!
cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 999")  # Нет такого ID
# Баланс не увеличен

# Результат: 100$ исчезли!
```

С транзакцией:

```
try:
    cursor.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
    cursor.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 999")
    conn.commit()
except:
    conn.rollback()  # Баланс возвращён
```

Слайд 5: Гонка данных (Race Condition)

Что такое гонка?

Два процесса одновременно изменяют одни данные.

Пример:

```
Время 0: User A видит balance = 100
Время 0: User B видит balance = 100

Время 1: User A: balance = 100 - 50 = 50
Время 1: User B: balance = 100 - 30 = 70

Время 2: User A записывает balance = 50
Время 2: User B записывает balance = 70

Результат: balance = 70 (должно быть 20!)
```

Слайд 6: Пример гонки - регистрация

Сценарий:

```
Курс: 10 мест

User A: Проверяет мест = 10 → OK
User B: Проверяет мест = 10 → OK

User A: Регистрируется → мест = 9
User B: Регистрируется → мест = 9

Результат: 11 человек на курсе!
```

Проблема:

- Проверка и действие не атомарны
- Между проверкой и действием состояние изменилось

Слайд 7: Решение 1 - Уникальность

Использование UNIQUE ограничения:

```
CREATE TABLE registrations (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    UNIQUE(user_id, course_id)  -- Один пользователь - один курс
);
```

При попытке дубликата:

```
try:
    cursor.execute(
        "INSERT INTO registrations (user_id, course_id) VALUES (%s, %s)",
        (user_id, course_id)
    )
    conn.commit()
except psycopg2.IntegrityError:
    conn.rollback()
    return "Already registered"
```

Слайд 8: Решение 2 - Атомарное изменение

UPDATE с условием:

```
UPDATE courses
SET seats = seats - 1
WHERE id = 1 AND seats > 0;
```

Проверка результата:

```
cursor.execute(
    "UPDATE courses SET seats = seats - 1 WHERE id = %s AND seats > 0",
    (course_id,)
)

if cursor.rowcount == 0:
    return "No seats available"
else:
    conn.commit()
    return "Registered"
```

Преимущества:

- Проверка и изменение атомарны
- Гарантируется, что seats > 0

Слайд 9: Решение 3 - SELECT FOR UPDATE

Блокировка строки:

```
try:
    # Начать транзакцию
    cursor.execute("BEGIN")

    # Заблокировать строку
    cursor.execute(
        "SELECT seats FROM courses WHERE id = %s FOR UPDATE",
        (course_id,)
    )
    row = cursor.fetchone()

    if row[0] <= 0:
        conn.rollback()
        return "No seats available"

    # Обновить
    cursor.execute(
        "UPDATE courses SET seats = seats - 1 WHERE id = %s",
        (course_id,)
    )

    conn.commit()
    return "Registered"

except Exception as e:
    conn.rollback()
    raise e
```

Слайд 10: Что такое индекс?

Индекс - структура данных для быстрого поиска

Аналогия:

- Без индекса: искать книгу в библиотеке перебирая все полки
- С индексом: искать по каталогу (алфавитный указатель)

Пример:

```
-- Без индекса
SELECT * FROM users WHERE email = 'john@example.com';
-- Полное сканирование таблицы (O(n))

-- С индексом
CREATE INDEX idx_users_email ON users(email);
SELECT * FROM users WHERE email = 'john@example.com';
-- Поиск по индексу (O(log n))
```

Слайд 11: Создание индексов

Синтаксис:

```
-- Простой индекс
CREATE INDEX idx_users_email ON users(email);

-- Составной индекс
CREATE INDEX idx_users_name_age ON users(name, age);

-- Уникальный индекс
CREATE UNIQUE INDEX idx_users_email ON users(email);

-- Удаление индекса
DROP INDEX idx_users_email;
```

Когда создавать индексы:

- Частые WHERE условия
- JOIN условия
- ORDER BY
- Частые уникальные проверки

Слайд 12: Влияние индексов на производительность

Чтение:

- ✅ Быстрее (в 10-1000 раз)
- ✅ Меньше чтения с диска

Запись:

- ❌ Медленнее (индекс нужно обновлять)
- ❌ Больше места на диске

Баланс:

```
-- Хорошо: часто читаем, редко пишем
CREATE INDEX idx_users_email ON users(email);

-- Плохо: часто пишем, редко читаем
CREATE INDEX idx_logs_timestamp ON logs(timestamp);
```

Слайд 13: EXPLAIN - анализ запросов

Проверка использования индекса:

```
EXPLAIN SELECT * FROM users WHERE email = 'john@example.com';
```

Результат без индекса:

```
Seq Scan on users  (cost=0.00..100.00 rows=1 width=100)
```

- Seq Scan = последовательное сканирование (медленно)

Результат с индексом:

```
Index Scan using idx_users_email on users  (cost=0.00..5.00 rows=1 width=100)
```

- Index Scan = поиск по индексу (быстро)

Слайд 14: Проблема N+1

Что такое N+1?

N запросов для получения связанных данных вместо 1.

Пример:

```
# Получить пользователей
users = session.query(User).all()  # 1 запрос

# Для каждого пользователя получить заказы
for user in users:
    orders = session.query(Order).filter_by(user_id=user.id).all()  # N запросов
    user.orders = orders

# Итого: 1 + N запросов
```

Проблема:

- 100 пользователей = 101 запрос
- Медленно!

Слайд 15: Решение N+1 - JOIN

Использование JOIN:

```
# Один запрос с JOIN
results = session.query(User, Order).join(Order).all()

users = {}
for user, order in results:
    if user.id not in users:
        users[user.id] = {'user': user, 'orders': []}
    users[user.id]['orders'].append(order)

# Итого: 1 запрос
```

SQLAlchemy eager loading:

```
# subqueryload
users = session.query(User).options(subqueryload(User.orders)).all()

# joinedload
users = session.query(User).options(joinedload(User.orders)).all()
```

Слайд 16: Решение N+1 - IN

Использование IN:

```
# Получить пользователей
users = session.query(User).all()  # 1 запрос

# Получить ID пользователей
user_ids = [u.id for u in users]

# Получить все заказы одним запросом
orders = session.query(Order).filter(Order.user_id.in_(user_ids)).all()  # 1 запрос

# Сгруппировать
orders_by_user = {}
for order in orders:
    if order.user_id not in orders_by_user:
        orders_by_user[order.user_id] = []
    orders_by_user[order.user_id].append(order)

# Итого: 2 запроса
```

Слайд 17: Составные индексы

Правило левого префикса:

```
CREATE INDEX idx_users_name_age ON users(name, age);
```

Используется:

```
WHERE name = 'John'              ✅
WHERE name = 'John' AND age = 30 ✅
WHERE age = 30                   ❌
```

Почему age не используется?

- Индекс упорядочен по name, затем age
- Поиск только по age требует полного сканирования

Решение:

```
CREATE INDEX idx_users_age ON users(age);
CREATE INDEX idx_users_name_age ON users(name, age);
```

Слайд 18: Частичный индекс

Индекс для подмножества данных:

```sql
-- Индекс только для активных пользователей
CREATE INDEX idx_users_active ON users(email) WHERE active = true;
```

Преимущества:

- Меньше места
- Быстрее обновление
- Полезен если часто фильтруем по active = true

Пример:

```sql
-- Использует индекс
SELECT * FROM users WHERE active = true AND email = 'john@example.com';

-- Не использует индекс
SELECT * FROM users WHERE active = false AND email = 'john@example.com';
```

Слайд 19: Практическое задание

Задание 1: Транзакция для перевода

- Реализовать перевод между счётами
- Использовать BEGIN/COMMIT/ROLLBACK
- Обработать ошибки

Задание 2: Гонка данных

- Создать сценарий с гонкой
- Продемонстрировать проблему
- Решить с SELECT FOR UPDATE

Задание 3: Индексы

- Создать индексы для частых запросов
- Проверить с EXPLAIN
- Измерить производительность

Задание 4: N+1 проблема

- Создать пример N+1
- Решить с JOIN
- Решить с IN

Задание 5: Оптимизация

Слайд 20: Вопрос для собеседования

Вопрос: Объясните, как избежать частичного сохранения и почему конкретный запрос может работать медленно.

Ожидаемый ответ:

1. Использовать транзакции (BEGIN/COMMIT/ROLLBACK)
2. При ошибке вызывать ROLLBACK
3. Медленный запрос может быть из-за:
    - Отсутствия индекса (проверить EXPLAIN)
    - N+1 проблемы (использовать JOIN или eager loading)
    - Полного сканирования таблицы (Seq Scan)
    - Отсутствия фильтрации
4. Решения: добавить индексы, оптимизировать запросы, использовать JOIN